JAMES DESIGN STUDIO

A Personal Scheduling & Portfolio Web Application for Graphic Design Businesses
Built using Python and Django Framework

JAMES EMMA EDEH

20231411222

Project Documentation
Covering: User Story, Use Case Diagram, Sequence Diagram, Class Diagram, and Deployment

Built with Python and the Django Framework

1. User Story

James Design Studio is a full-featured web application that enables a freelance graphic design studio to showcase creative works, automate client consultation bookings, calculate conflict-free real-time calendar slots, and manage production workflows online instead of through manual back-and-forth phone calls or unorganized messaging apps. The system supports three primary roles: a **Customer / Client** who browses portfolio designs and books design consultation sessions, a **Designer / Staff Member** who manages availability, reviews design briefs, and updates project statuses, and a **Business Admin** who oversees the design service catalog, pricing in Naira (■), and customer accounts. The user stories below describe the system from the perspective of each user role.

1.1 Customer Stories

- **As a customer**, I want to browse high-resolution portfolio designs (event flyers, brand identities, infographics, social packs) with uncropped previews, so that I can evaluate the designer's style and quality before committing.
- **As a customer**, I want to see the studio's curated service packages with transparent Naira (■) pricing and durations, so that I know the financial investment upfront.
- **As a customer**, I want to choose a date and pick from real-time available time slots that automatically exclude booked sessions, so that I do not waste time selecting an unavailable slot.
- **As a customer**, I want to provide my design brief, reference links, and contact details in a guided 4-step wizard, so that the designer understands my project requirements beforehand.
- **As a customer**, I want to instantly receive a booking reference code and download an .ICS calendar invite, so that I can add the discovery meeting to Google Calendar or Apple Calendar.
- **As a customer**, I want to look up my booking status, reschedule, or cancel my appointment online, so that I have full control over my schedule without making phone calls.

1.2 Staff / Designer Stories

- **As a designer**, I want to configure my recurring weekly working hours and lunch break intervals (e.g., Monday–Friday 9:00 AM to 5:00 PM), so that clients can only book me during active working hours.
- **As a designer**, I want to add specific blackout dates (holidays, illness, or studio vacations), so that the slot calculation engine prevents bookings on those dates.
- **As a designer**, I want a centralized dashboard showing key metrics (total bookings, confirmed Naira revenue, pending reviews), so that I have full business clarity.
- **As a designer**, I want to view all appointments in an organized table and update statuses (Pending, Confirmed, In Progress, Completed, Cancelled), so that my client pipeline is accurate.
- **As a designer**, I want to export my appointments list to CSV, so that I can perform accounting and archive client records.

1.3 Business Admin Stories

- **As a business admin**, I want to add, edit, or deactivate service packages and adjust prices in Naira (■), so that our public offerings remain up-to-date.
- **As a business admin**, I want to upload, reorder, and feature new graphic design artworks in the portfolio showcase, so that visitors always see recent high-impact works.

- **As a business admin**, I want to manage client reviews and testimonials from the Django administration panel, ensuring authentic client feedback is showcased.

1.4 Non-Functional Expectations

- **Concurrency & Anti-Collision:** The system must strictly prohibit double-booking. The slot calculation engine and model-level validation must prevent two clients from reserving overlapping time slots.
- **Responsive & Editorial UI:** The interface must deliver a modern, high-contrast editorial aesthetic across mobile phones, tablets, and wide desktop screens.
- **Security & Access Control:** Administrative dashboards, appointment modifications, and working hours settings must require authentication, protecting sensitive client data.
- **Automated Calendar Sync:** The system must generate standard iCalendar (.ics) RFC-5545 payloads for universal cross-platform calendar integration.

2. Use Case Diagram

The diagram below illustrates the interactions between the system actors (Visitor, Customer/Client, Designer, and Business Admin) and the primary functional capabilities of the web application.

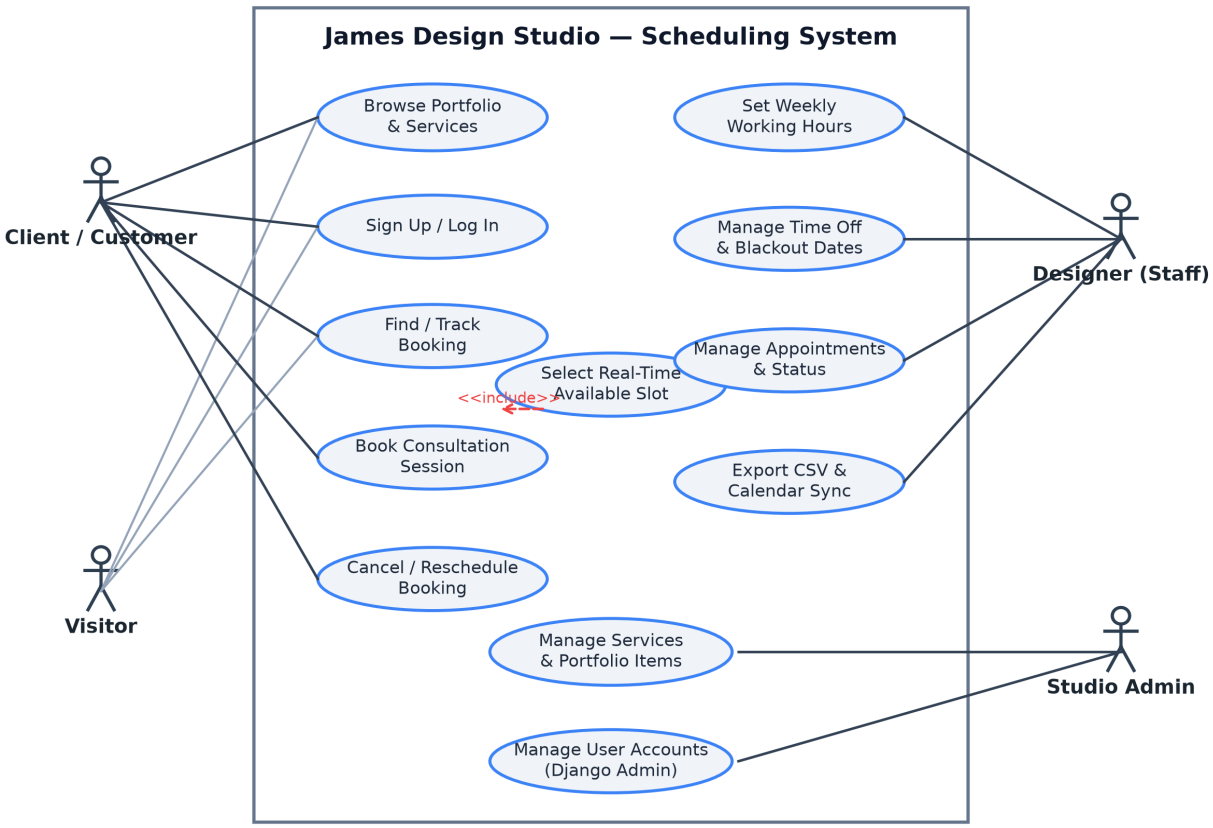

Figure 1: Use Case Diagram for James Design Studio Scheduling Web Application

2.1 Actors

- **Visitor:** An unauthenticated user browsing portfolio artworks, service catalog packages, and studio contact information.
- **Customer / Client:** A client who selects a design package, provides a design brief, selects a real-time calendar slot, and manages their appointment.
- **Designer (Staff):** The creative professional who defines working schedules, reviews client briefs, manages appointment lifecycle statuses, and exports scheduling data.
- **Business Admin:** The studio administrator with full access to the Django administration panel for managing services, portfolio items, testimonials, and user permissions.

2.2 Detailed Use Case Descriptions

Use Case: Book Design Consultation Session

Field	Description
Actor	Customer / Client
Description	Enables a client to choose a design service, input project requirements, pick a conflict-free slot from the live calendar, and receive a confirmed booking reference.
Precondition	At least one active design service and active working hours schedule exist in the database.
Main Flow	1. Client opens booking wizard (/schedule/book/). 2. Client selects design service package (e.g., Brand Identity Suite). 3. Client selects desired date from interactive calendar. 4. System queries database via AJAX (/schedule/api/available-slots/) and dynamically renders available time slots. 5. Client selects a slot and fills in contact details, project deadline, meeting preference, and design brief. 6. Client submits the booking form. 7. System validates slot availability, generates unique booking reference (e.g. DES-102938), saves record as 'PENDING', and renders confirmation page with .ICS calendar download.
Alternate Flow	If the selected time slot was taken concurrently by another user, the system rejects the submission with a clear collision warning and prompts the client to choose another time.
Postcondition	A new Appointment record is created in the database and is immediately visible on the designer's dashboard.
Includes	Select Real-Time Available Slot, Re-check Anti-Collision Validation.

Use Case: Set Weekly Working Hours & Blackout Dates

Field	Description
Actor	Designer (Staff)
Description	Allows the designer to define recurring working hours per weekday, configure lunch breaks, and block out holiday dates.
Precondition	The designer is authenticated with staff/designer credentials.
Main Flow	1. Designer logs into dashboard and navigates to Availability Settings (/dashboard/settings/working-hours/). 2. Designer sets daily start time, end time, break intervals, and 'Day Off' toggle for Monday through Sunday. 3. Designer optionally adds blackout dates with reasons (e.g., Studio Vacation). 4. System validates time sequences (start < end) and commits configuration to database.
Postcondition	Customer slot calculations immediately reflect the updated operating hours and blacked-out periods.

Use Case: Manage Service Catalog & Portfolio Works

Field	Description
Actor	Business Admin
Description	Allows the studio administrator to add, edit, reorder, or deactivate design packages and portfolio showcase items.
Precondition	User is authenticated with superuser / admin privileges.
Main Flow	1. Admin opens Django admin (/admin/). 2. Admin creates/edits a Service with name, duration, buffer time, and price in Naira (■). 3. Admin uploads portfolio images, configures categories, and sets featured display order. 4. System saves records and refreshes public catalog and portfolio grid immediately.
Postcondition	Public website displays updated packages, Naira figures, and artwork showcase.

3. Sequence Diagram

The Book Consultation Session workflow represents the core interaction of the system. It coordinates client input across a multi-step interface, asynchronous JSON slot calculations, double-booking prevention, database transactions, and calendar file generation.

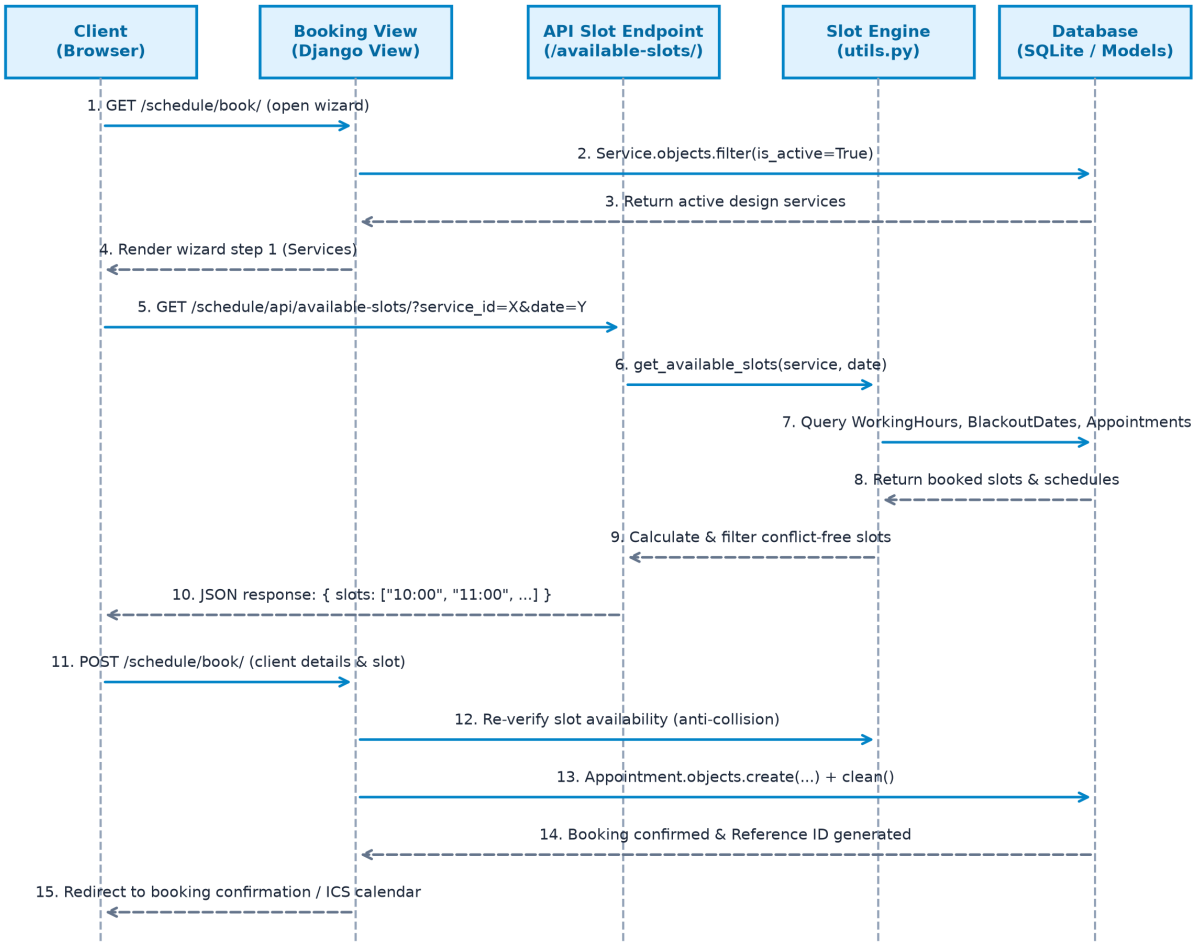

Figure 2: Sequence Diagram for the Booking Workflow & Real-Time Slot Engine

3.1 Description of the Flow

- **1–4. Service Package Selection:** The client navigates to the booking wizard (/schedule/book/). The Django view queries active services from the database and renders Step 1 with durations and Naira pricing.
- **5–6. Interactive Date Selection:** When the client selects a service and picks a calendar date, the front-end JavaScript issues an asynchronous HTTP GET request to /schedule/api/available-slots/?service_id=X&date;=Y.
- **7–9. Slot Engine Calculation:** The API endpoint executes get_available_slots() in utils.py. The engine loads the designer's WorkingHours for that weekday, verifies that the date is not in BlackoutDates, and retrieves all existing appointments for that date.

- **10. Dynamic Slot Rendering:** The engine computes non-overlapping time slots (accounting for service duration and buffer time) and returns a JSON payload. The client UI renders selectable time pills.
- **11–13. Submission & Double-Booking Prevention:** The client enters their design brief, deadline, meeting type, and submits the form via HTTP POST. Before committing, the server re-runs the availability algorithm to prevent race conditions. The Appointment model clean() method executes to verify time boundaries.
- **14–15. Confirmation & Calendar Payload:** The record is saved with status 'PENDING', generating a reference ID (DES-XXXXXX). The user is redirected to the confirmation view where an RFC-5545 iCalendar (.ics) invite is generated for download.

4. Class Diagram

The diagram below depicts the data model architecture of James Design Studio, illustrating model fields, methods, and relationships spanning authentication, scheduling, portfolio presentation, and client reviews.

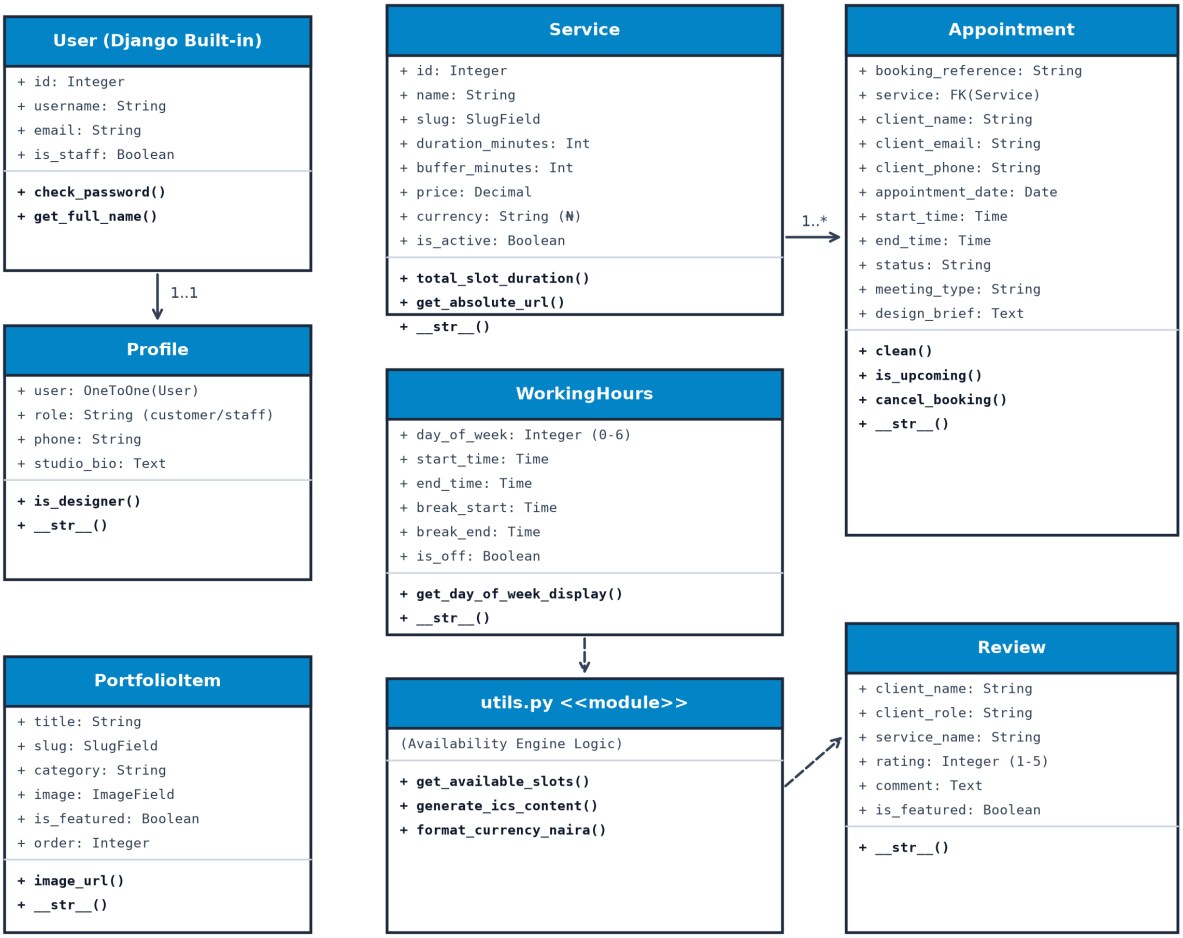

Figure 3: Class Diagram of Models and Architecture

4.1 Description of Classes & Modules

- **User (Django Built-in):** Django's native authentication model storing username, hashed password, email, and staff authorization flags. Acts as the root anchor for account relationships.
- **Profile:** Extends User via a One-to-One relationship with role designations ('customer', 'staff', 'admin'), direct phone number, and studio biography. Created automatically via Django post_save signals.
- **Service:** Represents a design offering (e.g. Brand Identity Suite, Event Flyer Design). Stores service duration in minutes, buffer padding time, active boolean status, and price formatted in Naira (₦).

- **WorkingHours:** Encapsulates daily recurring schedule blocks (days 0–6 for Monday through Sunday), start time, end time, and break periods, defining the designer's weekly availability envelope.
- **BlackoutDate:** Represents specific single dates or multi-day leaves during which all slot generations are blocked (e.g., public holidays, studio retreats).
- **Appointment:** The core transactional entity linking a client, design service, chosen date, start/end time range, design brief text, brand asset URLs, and status ('PENDING', 'CONFIRMED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED'). Enforces strict overlapping validation in clean().
- **PortfolioItem:** Manages visual gallery works, categories (Branding, Flyers, Infographics, Social Media), image assets, client names, and display ordering on the homepage 4x2 grid.
- **Review:** Stores client testimonials, ratings (1–5 stars), feedback comments, and featured status displayed in the studio reviews section.
- **utils.py (Availability & Calendar Engine):** Contains business logic functions get_available_slots() and generate_ics_content() that process calendar availability and format iCalendar exports.

5. Hosted Link & Deployment Details

James Design Studio is deployed to production using a modern serverless cloud architecture on **Vercel**, integrated with a continuous deployment pipeline tracking the **GitHub** repository.

5.1 Production Deployment Architecture

- **Cloud Platform:** Vercel Serverless Python 3.12 Runtime.
- **Static Asset Handling:** WhiteNoise middleware (whitenoise.middleware.WhiteNoiseMiddleware) with gzip/brotli compression and cached header routing.
- **Continuous Deployment (CI/CD):** Every commit pushed to the GitHub main branch triggers an automatic build, dependency installation, static collection, and atomic zero-downtime deployment.

5.2 Project Links & Credentials

Resource	Link / Value
Live Production URL	https://graphic-design-scheduler-roan.vercel.app
GitHub Repository	https://github.com/joebthebest/graphic-design-scheduler
Google Submission Form	https://forms.gle/peLot6q8PS8qUzGU7
Designer / Admin Username	**admin**
Designer / Admin Password	**admin123**
Designer Portal URL	https://graphic-design-scheduler-roan.vercel.app/accounts/login/
Django Admin Panel	https://graphic-design-scheduler-roan.vercel.app/admin/

5.3 Steps to Replicate / Deploy

1. **Clone Repository:** git clone https://github.com/joebthebest/graphic-design-scheduler.git
2. **Install Dependencies:** pip install -r requirements.txt
3. **Run Database Migrations:** python manage.py migrate
4. **Seed Initial Data:** python manage.py seed_data (Initializes services in ■, working hours, portfolio artworks, and admin user).
5. **Run Automated Tests:** python manage.py test (Executes 7 unit tests covering models, slot algorithms, views, and security).
6. **Start Local Server:** python manage.py runserver 127.0.0.1:8000
7. **Deploy on Vercel:** Connect repository on vercel.com/new and deploy with zero additional configuration.